

Ch5 Algorithm

演算法 Algorithm

非正式定義：表示執行步驟，並最終滿足某個目標的要求

正式定義：一組有序、明確、可執行的步驟且定義終止過程

演算法是抽象的，可以有不同表示方式。

就像同一個故事，翻譯成不同語言、電影...情節還是一樣。

Algorithm、Process、Program

Program 是表達 Algorithm的一種形式

Process 是執行 Program的一種活動實體

→ Process 是執行 Algorithm的一種活動實體

pseudocode 虛擬碼

相較於程式語言更容易理解、標準更寬鬆

相較於自然語言更精確

Assignment

name = expression

Selection

```
if (condition):  
    activity  
else:  
    activity
```

Nested 巢狀

```
if (not raining):  
    if (temperature == hot):  
        go swimming  
    else:  
        play golf  
else:  
    watch television
```

Functions

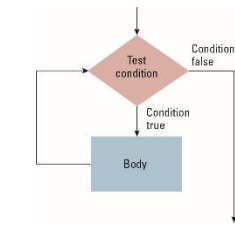
Algorithms 可以結合起來，形成新的 Algorithm

```
if (. . .):  
    ProcessLoan()  
else:  
    RejectApplication()
```

Loop 迴圈 (Iteration 迭代)

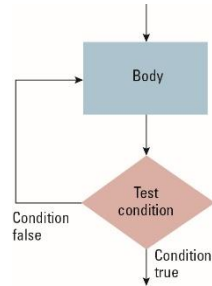
1. PreTest

```
while (condition):  
    body
```



2. PostTest

```
repeat:  
    body  
until(condition)
```



Recursive 遞迴

- 初始化 (initialization): 通常是第一次呼叫，定義初始參數。
- 修改 (modification): 在每次遞迴呼叫中，傳入不同的 (通常更接近終止條件的) 參數。
- 終止測試 (base case): 當達到基準情況時停止遞迴，避免無限呼叫。

Polya's Problem Solving Steps

1. 理解問題
2. 制定解決計畫
3. 執行解決計畫
4. 評估解決方法的正確性與解決其他問題的潛力

解決問題的核心：取得突破口「把腳伸進門縫 Get your foot in the door」

- 反向思考：從目標倒過來找線索，拆開成品，理解它形成的方式
- 找相關的問題：先從簡單案例著手，找到通用的原則，再抽象化成解法

逐步求精 Stepwise Refinement

- 把複雜問題拆成幾個子問題，再層層細節化
- Top-down methodology 由上而下的解法 ⇔ Bottom-up methodology 由小部分往上組成複雜系統
- 現代開發通常兩種搭配

Binary Search 二分搜

- 條件：資料已排序

- 做法：

每次找中間值

比對目標值

決定向左或向右，每次搜尋範圍砍半

- $\Theta(\log n)$

Insertion Sort 插入排序法

- 做法：

把「當前的元素」插入左列已排序的陣列中

- $\Theta(n^2)$

Assertion(斷言)

執行過程中，測試某個應該恆真的條件是否正確

- Preconditions
- Loop invariants：迴圈中執行前、每一次執行、執行後都應該成立的條件，通常可以代表迴圈的目的

5-6 我暫時懶得寫